

Das Korn-Spiel

Vorschlag für ein Simulationsspiel zur Dreigliederung des sozialen Ökosystems

Arbeitsentwurf, September 2026. Datenmodelle lauffähig und getestet (Python 3.11+, Pydantic 2).

Worum geht es?

Eine Stadt, die nach der Gewaltenteilung 2.0 lebt. Kultursphäre, Politik und Recht und Wirtschaftsleben stehen nebeneinander, keine regiert die andere. Beahlt wird mit Korn: einer Währung, die altert, an der nachwachsenden Naturgrundlage hängt und nie in Euro zurückgetauscht wird.

Das Ganze ist Simulation und Spiel zugleich. Man kann zuschauen, wie Bots die Stadt bewegen. Oder man übernimmt selbst eine Rolle.

Das Ziel ist nicht, die Dreigliederung zu erklären. Das Ziel ist, dass man sie erlebt. Am deutlichsten dann, wenn man sie verletzt und sieht, was daraus folgt.

Warum nicht einfach SimCity?

Weil SimCity ein Gesellschaftsbild eingebaut hat. Ein Bürgermeister zoniert, setzt Steuern fest, verteilt das Budget. Die Stadt reagiert darauf wie ein Automat.

Das ist der Einheitsstaat. Politik regiert in Wirtschaft und Kultur hinein, und das nicht als Fehler, sondern als Spielprinzip.

Im Korn-Spiel gibt es niemanden, der das Ganze steuert. Es gibt Organe mit begrenzter Zuständigkeit, und die Stadt entsteht aus dem, was Menschen in ihnen tun.

Welche Rollen gibt es?

Jeder Mensch ist **Bürger**. Immer. Man arbeitet, kauft, schenkt, stimmt ab. Diese Grundrolle hat jeder Spieler.

Dazu kommen drei Organ-Rollen, eine je Sphäre:

- **EVG** (Erzeuger-Verwender-Gemeinschaft): Man verhandelt zwischen Erzeugern, Händlern und Verwendern die Preise. Maßstab ist der gerechte Preis.
- **Politik und Recht**: Man bereitet Rahmenregeln vor, etwa zu Arbeitszeit oder Bodenrecht. Beschließen tun alle Bürger, mit Mehrheit. Preise und Inhalte sind nicht Sache dieses Organs.
- **Kultursphäre**: Man führt eine Schule, eine Forschungsstätte, eine Kunstwerkstatt. Sie lebt von Schenkgeld und muss die Bürger überzeugen, ihr sterbendes Korn ihr zu geben.

Jede Rolle kann ein Mensch spielen oder ein Bot. Unbesetzte Rollen übernimmt die Simulation. Und jederzeit kann man wechseln.

Warum hat eine Person mehrere Mitgliedschaften?

Weil die drei Sphären keine drei Gruppen von Menschen sind. Sie sind drei Beziehungen jedes Menschen.

Dora ist Lehrerin an der freien Schule, kauft Brot als Verwenderin in der EVG und stimmt in der Rechtsgemeinschaft ab. Alles gleichzeitig.

Deshalb hat im Datenmodell jede Person beliebig viele, sich überlappende Mitgliedschaften, aber genau eine Rolle, die sie gerade spielt.

Wie läuft eine Runde ab?

Eine Runde ist eine Jahreszeit. Korn altert in Jahren, Spielzüge dauern Minuten. Rundenbasiert passt das zusammen.

Jede Runde hat sechs Phasen:

1. **Natur:** Die Welt erntet, lässt Korn altern, frischt an den Naturbetrieben auf.
2. **Absichten:** Menschen und Bots sagen, was sie wollen.
3. **Verhandlung:** Die EVG einigen sich auf Preise.
4. **Abstimmung:** Die Rechtsgemeinschaft entscheidet über Anträge.
5. **Auflösung:** Die Welt prüft, bucht und erkennt Übergriffe.
6. **Rückschau:** Kennzahlen und Folgen werden für alle sichtbar.

Wichtig: Spieler äußern nur Absichten. Ob etwas gilt, entscheidet allein die Welt. Das macht das Spiel betrugssicher und die Bots austauschbar.

Was macht das Spiel spannend?

Die Konflikte zwischen den Rollen. Wirtschaftssimulationen werden schnell zu Tabellenkalkulationen. Spannung entsteht erst, wenn Menschen mit verschiedenen Aufgaben um dieselben knappen Dinge ringen.

Und die Übergriffe. Sie sind erlaubt. Wer als Politik und Recht einen Brotpreis per Beschluss festsetzt, darf das. Ein paar Runden später zeigt die Simulation, was passiert: Der Bäcker kann nicht mehr leben, das Brot wird knapp.

Man lernt die Gewaltenteilung 2.0 nicht durch Belehrung, sondern durch Folgen.

Wie erkennt die Welt einen Übergriff?

Hier liegt die schönste Eigenschaft des Modells. Es gibt genau drei *gestaltende* Handlungen, eine je Sphäre: Preis vorschlagen, Regel beantragen, Inhalt bestimmen.

Und es gibt drei Organ-Rollen. Rolle mal Handlung ergibt eine 3×3-Matrix. Die Diagonale ist gesund, die sechs Zellen daneben sind Übergriffe. Die Übergriffs-Matrix entsteht also nicht als Zusatzregel, sie fällt aus der Struktur heraus.

handelt als ...	Preis vorschlagen	Regel beantragen	Inhalt bestimmen
EVG	gesund	Wirtschaft → Recht: Gesetze nach Marktinteresse	Wirtschaft → Kultur: gekaufte Bildung
Politik und Recht	Recht → Wirtschaft: Preis per Beschluss	gesund	Recht → Kultur: Staatslehrplan
Kultursphäre	Kultur → Wirtschaft: Weltanschauung diktiert Preise	Kultur → Recht: Bekenntnis wird Gesetz	gesund

Die Beschriftung der Zellen ist eine Beispielbelegung. Das letzte Wort hat die Doppelmatrix.

Bürgerhandlungen greifen nie über. Wer als Bürgerin eine Regel beantragt, handelt aus dem Rechtsleben selbst. Dass die Wirtschaft ihre Interessen über Bürger einschleusen kann, ist dabei kein Fehler im Modell, sondern Realismus. Genau so funktioniert Lobbyarbeit. Ob das Spiel solche verdeckten Übergriffe erkennen soll, ist eine offene Frage.

Woran merkt man, ob die Stadt gesund ist?

Nicht an einem Punktestand. An Wahrnehmungsorganen, die für alle sichtbar sind:

- **Deckung des Lebensunterhalts:** Kann jeder leben?
- **Abweichung vom gerechten Preis:** Wie weit liegen die EVG-Preise vom Maßstab der Urzelle entfernt?
- **Geldmenge:** Sie pendelt sich auf dem Umlauf ein, nicht auf dem Startwert.
- **Schenkgeld zur Kultursphäre:** Kommt genug an?
- **Fristen-Mismatch:** Wie lange warten Vorhaben auf Korn mit passender Laufzeit?
- **Übergriffe:** Wie oft, in welcher Zelle der Matrix?
- **Beteiligung an Abstimmungen:** Lebt das Rechtsleben?

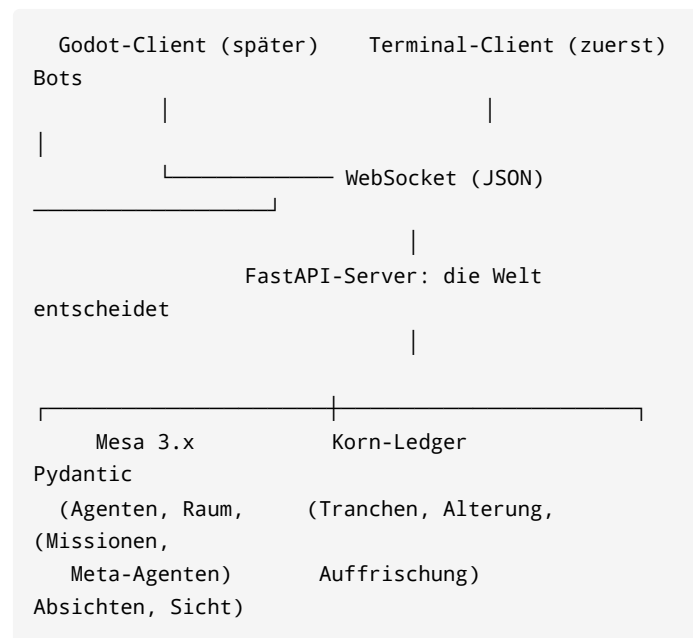
Woher kommt der gerechte Preis?

Aus Steiners Urzelle des Wirtschaftslebens (GA 340): Ein Erzeuger soll für sein Erzeugnis so viel erhalten, dass er seine Bedürfnisse decken kann, bis er ein gleiches Erzeugnis wieder hergestellt hat.

Das lässt sich direkt rechnen. Ben braucht 1000 Halme am Tag, backt an einem Tag 50 Brote. Gerechter Preis: 20 Halme je Brot.

Die EVG verhandelt frei. Der gerechte Preis steht nur als Maßstab daneben. Die Abweichung davon ist die wichtigste Kennzahl des Spiels.

Wie sieht die Technik aus?



Mesa trägt Agenten und Raum. Es kennt überlappende Mitgliedschaften in Meta-Agenten, genau das, was die EVG, die Rechtsgemeinschaft und die Kulturstätten brauchen. Die Browser-Ansicht bringt Mesa selbst mit. Mit Mesa-Geo ließe sich der Kiez auf der echten Karte simulieren. Weil Mesa 4 in Arbeit ist, wird auf Version 3.x festgelegt.

Der Korn-Ledger ist das einzige wirklich neue Stück Code:

Tranchen mit Ablaufdatum, nicht fungibel. Er wird aus `korn-sim` übernommen.

Pydantic prüft alles, was von außen kommt: Missionen aus YAML, Absichten aus dem Netz, die Sicht, die an Spieler geht.

Godot kommt erst, wenn das Spiel im Terminal trägt.

Die Datenmodelle hängen bewusst nicht von Mesa ab. Mesa-Agenten tragen die Datensätze, nicht umgekehrt. So übersteht das Modell auch einen Wechsel der Engine.

Wie sehen die Datenmodelle aus?

Die folgenden Abschnitte stammen unverändert aus `modelle.py`. Zwölf Tests prüfen, dass sie tragen.

Was sind die Grundbegriffe?

Gerechnet wird in Halmen, ganzzahlig. Ein Korn sind hundert Halme. So bleiben Summen exakt.

```
HALME_JE_KORN = 100 # 1 Korn = 100 Halme; intern
wird nur ganzzahlig gerechnet
TAGE_JE_RUNDE = 91 # eine Runde = eine Jahreszeit

Halme = Annotated[int, Field(ge=0)]
Id = Annotated[str, Field(min_length=1,
pattern=r"^[a-z0-9][a-z0-9_\-]*$")]

class Sphaere(str, Enum):
    """Die drei Glieder des sozialen Ökosystems
    (Gewaltenteilung 2.0)."""
```

```

KULTUR = "kultur"          # Kultursphäre
RECHT = "recht"            # Politik und Recht
WIRTSCHAFT = "wirtschaft"  # Wirtschaftsleben

class Modell(BaseModel):
    """Basis: unbekannte Felder sind ein Fehler,
    keine stille Überraschung."""

    model_config = ConfigDict(extra="forbid")

```

Wie wird Korn gebucht?

Korn ist nicht fungibel. Zwei Einheiten mit gleichem Betrag, aber verschiedenem Ablaufdatum sind verschiedene Dinge. Deshalb gibt es keine Kontostände, sondern Tranchen. Die Tranche ist eine schlanke dataclass, weil der Ledger sehr viele davon bewegt.

```

class Verwendung(str, Enum):
    """Kein Sortenmerkmal des Geldes, sondern der
    Zustand, in dem es gebraucht wird."""

    KAUF = "kauf"
    LEIH = "leih"
    SCHENK = "schenk"

    @dataclass(slots=True)
    class Tranche:
        """Eine Korn-Tranche. Korn ist nicht fungibel:
        Betrag UND Ablaufdatum zählen."""

        id: int
        halme: int
        ablauf_tag: int
        halter: str
        region: str

```

```

        verwendung: Verwendung = Verwendung.KAUF
        geboren_tag: int = 0

    def restlaufzeit(self, tag: int) -> int:
        return self.ablauf_tag - tag

class KornRegeln(Modell):
    """Alle Stellschrauben des Protokolls an einem
    Ort."""

    frist_jahre: float = Field(20, gt=0)
    auffrischung_ab_jahren: float = Field(1, gt=0)
    schenkgrenze_tage: int = Field(90, ge=0)
    emission: Literal["nur_auffrischung",
    "ertragsgedeckt"] = "nur_auffrischung"
    emission_je_ertrag: float = Field(0.0, ge=0)

    @model_validator(mode="after")
    def _plausibel(self) -> "KornRegeln":
        if self.auffrischung_ab_jahren >=
self.frist_jahre:
            raise ValueError("Auffrischung muss vor
Ablauf der Frist beginnen")
        if self.schenkgrenze_tage >=
self.frist_jahre * 365:
            raise ValueError("Schenkgrenze länger
als die ganze Frist")
        if self.emission == "ertragsgedeckt" and
self.emission_je_ertrag == 0:
            raise ValueError("ertragsgedeckte
Emission braucht emission_je_ertrag > 0")
        return self

```

Wie sieht eine Region aus?

Jede Region hat ihre Naturgrundlage als Portfolio. Ihr Index steuert die Kaufkraft. Die Euro-Stelle ist die Membran zur

Außenwelt. Über die Rechenstelle fließt jede Region in die eine Weltrechnung.

```
class Naturgut(Modell):
    """Ein Posten der Naturgrundlage: der
    nachwachsende Teil, durch Arbeit gewonnen."""

    name: Id
    normalertrag: float = Field(gt=0)
    gewicht: float = Field(1.0, gt=0)
    einheit: str = "t"

class Naturgrundlage(Modell):
    """Das regionale Portfolio. Sein Index steuert
    die Kaufkraft, nicht die Geldmenge."""

    gueter: list[Naturgut] = Field(min_length=1)

    def index(self, ertraege: dict[str, float]) ->
float:
        summe = sum(g.gewicht for g in self.gueter)
        return sum(
            g.gewicht * ertraege.get(g.name,
g.normalertrag) / g.normalertrag
            for g in self.gueter
        ) / summe

class EuroStelle(Modell):
    """Die Membran zur Außenwelt. Euro nie nach
    innen, Korn nie nach außen.

    Euro kommen nur als eingebrachte
    Produktionsmittel herein. Mitglieder zahlen
    hier Korn, die Stelle bestellt draußen in Euro -
    und umgekehrt.
    """
```

```

    id: Id
    euro_vorrat: float = Field(0.0, ge=0)

class Region(Modell):
    id: Id
    name: str
    naturgrundlage: Naturgrundlage
    euro_stelle: EuroStelle | None = None
    rechenstelle: Id = "welt" # regionale
    Portfolios fließen in die Weltrechnung

```

Wer spielt mit?

Personen mit überlappenden Mitgliedschaften und genau einer gespielten Rolle. Wer eine Organ-Rolle spielen will, muss Mitglied eines Organs dieser Sphäre sein.

```

class Steuerung(str, Enum):
    MENSCH = "mensch"
    BOT = "bot"

class Rolle(str, Enum):
    """Aus welcher Kraft heraus eine Person gerade
    handelt."""

    BUEGER = "buerger" # jede Person, immer
    EVG = "evg" # als Organ des
    Wirtschaftslebens
    RECHT = "recht" # als Organ von Politik und
    Recht
    KULTUR = "kultur" # als Organ der
    Kultursphäre

ROLLEN_SPHAERE: dict[Rolle, Sphaere | None] = {
    Rolle.BUEGER: None,

```

```

    Rolle.EVG: Sphaere.WIRTSCHAFT,
    Rolle.RECHT: Sphaere.RECHT,
    Rolle.KULTUR: Sphaere.KULTUR,
}

class Mitgliedschaft(Modell):
    """Eine Beziehung zu einem Organ.
    Mitgliedschaften dürfen sich überlappen."""

    organ: Id
    sphaere: Sphaere
    funktion: str = "mitglied" # z. B. erzeuger,
    verwender, lehrerin, antragstellerin

class Beduerfnisse(Modell):
    """Aufbauphase: erst der Lebensunterhalt, dann
    die Arbeitsmittel in Korn."""

    lebensunterhalt_je_runde: Halme
    arbeitsmittel_je_runde: Halme = 0

    @property
    def je_tag(self) -> float:
        return (self.lebensunterhalt_je_runde +
        self.arbeitsmittel_je_runde) / TAGE_JE_RUNDE

class Person(Modell):
    """Ein Mensch - gespielt oder simuliert.

    Die drei Sphären sind keine drei Gruppen von
    Menschen, sondern drei
    Beziehungen jedes Menschen. Deshalb hat eine
    Person Mitgliedschaften
    in allen Sphären zugleich, aber genau eine
    Rolle, die sie gerade spielt.
    """

```

```

id: Id
name: str
region: Id
steuerung: Steuerung = Steuerung.BOT
spielt_rolle: Rolle = Rolle.BUERGER
mitgliedschaften: list[Mitgliedschaft] = []
beduerfnisse: Beduerfnisse
faehigkeiten: list[str] = []

@model_validator(mode="after")
def _rolle_braucht_organ(self) -> "Person":
    sphaere = ROLLEN_SPHAERE[self.spielt_rolle]
    if sphaere and not any(m.sphaere is sphaere
for m in self.mitgliedschaften):
        raise ValueError(
            f"{self.id}: Rolle
{self.spielt_rolle.value} ohne Mitgliedschaft "
            f"in einem Organ der Sphäre
{sphaere.value}")
    )
    return self

```

Welche Organe gibt es?

Vier Arten: EVG und Naturbetrieb im Wirtschaftsleben, die Rechtsgemeinschaft in Politik und Recht, die Kulturstätte in der Kultursphäre. Die Zwanziger-Regel für EVG ist als Schalter eingebaut, nicht als Dogma.

```

class Organ(Modell):
    id: Id
    name: str
    region: Id

class EVG(Organ):
    """Erzeuger-Verwender-Gemeinschaft: findet

```

```
Preise, setzt sie nicht per Macht."""
```

```
    sphaere: Literal[Sphaere.WIRTSCHAFT] =  
Sphaere.WIRTSCHAFT  
    produkte: list[Id] = Field(min_length=1)  
    erzeuger: list[Id] = Field(min_length=1)  
    haendler: list[Id] = []  
    verwennder: list[Id] = Field(min_length=1)  
    max_mitglieder: int = Field(20, gt=0) #  
Zwanziger-Regel als Schalter  
  
    @property  
    def mitglieder(self) -> set[str]:  
        return set(self.erzeuger) |  
set(self.haendler) | set(self.verwennder)  
  
    @model_validator(mode="after")  
    def _groesse(self) -> "EVG":  
        if len(self.mitglieder) >  
self.max_mitglieder:  
            raise ValueError(f"{self.id}: mehr als  
{self.max_mitglieder} Mitglieder")  
        return self
```

```
class Naturbetrieb(Organ):  
    """Der einzige Ort, an dem Korn entsteht oder  
aufgefrischt wird."""
```

```
    sphaere: Literal[Sphaere.WIRTSCHAFT] =  
Sphaere.WIRTSCHAFT  
    naturgut: Id  
    betreiber: list[Id] = Field(min_length=1)  
    anerkannt: bool = False
```

```
class Rahmenregel(Modell):  
    """Was Politik und Recht beschließen darf:  
Rahmen, keine Preise, keine Inhalte."""
```

```

    id: Id
    titel: str
    gegenstand: Literal["arbeitszeit", "bodenrecht",
"schutz", "verfahren", "sonstiges"]
    werte: dict[str, float | str] = {}
    beschlossen_runde: int | None = None

class Rechtsgemeinschaft(Organ):
    """Alle mündigen Menschen der Region,
gleichberechtigt. Das Organ bereitet vor."""

    sphaere: Literal[Sphaere.RECHT] = Sphaere.RECHT
    regeln: list[Rahmenregel] = []
    mehrheit: float = Field(0.5, gt=0, le=1)

class Kulturstaeette(Organ):
    """Schule, Forschung, Kunst, Heilkunst - lebt
von Schenkgeld."""

    sphaere: Literal[Sphaere.KULTUR] =
Sphaere.KULTUR
    art: Literal["schule", "forschung", "kunst",
"heilkunst", "sonstiges"]
    mitwirkende: list[Id] = Field(min_length=1)
    bedarf_je_runde: Halme

```

Wie entsteht ein Preis?

Produkte kennen ihre Herstellungsdauer. Daraus rechnet die Urzelle den gerechten Preis. Die Preisvereinbarung trägt ihn als Maßstab mit.

```

class Produkt(Modell):
    id: Id
    name: str

```

```

        herstellungsdauer_tage: int = Field(gt=0)
        menge_je_herstellung: float = Field(gt=0)
        einheit: str = "stück"

def gerechter_preis(erzeuger: Person, produkt:
Produkt) -> int:
    """Urzelle des Wirtschaftslebens (GA 340), als
    Rechenregel.

    Der Erzeuger soll für sein Erzeugnis so viel
    erhalten, dass er seine
    Bedürfnisse decken kann, bis er ein gleiches
    Erzeugnis wieder hergestellt hat.
    Ergebnis in Halmen je Einheit, aufgerundet.
    """
    bedarf = erzeuger.beduerfnisse.je_tag *
produkt.herstellungsdauer_tage
    return math.ceil(bedarf /
produkt.menge_je_herstellung)

class Preisvereinbarung(Modell):
    """Ergebnis einer EVG-Verhandlung. Der gerechte
    Preis steht als Maßstab daneben."""

    evg: Id
    produkt: Id
    preis_je_einheit: Halme
    gerechter_preis: Halme
    gueltig_ab_runde: int = Field(ge=0)

    @property
    def abweichung(self) -> float:
        """Relative Abweichung vom gerechten Preis:
        0 = gerecht, <0 = Erzeuger verarmt."""
        if self.gerechter_preis == 0:
            return 0.0

```

```
        return self.preis_je_einheit /  
        self.gerechter_preis - 1.0
```

Was kann man tun?

Alles ist eine Absicht. Jede Absicht kennt ihre Zielsphäre, sofern sie eine Sphäre gestaltet. Ein Client schickt pro Runde ein Paket davon an den Server.

```
class Absicht(Modell):  
    """Was jemand will. Ob es gilt, entscheidet  
    allein die Welt (der Server).  
  
    `ziel_sphaere` ist gesetzt, wenn die Handlung  
    eine Sphäre gestaltet.  
    Bürgerhandlungen (zahlen, schenken, abstimmen)  
    gestalten keine - der  
    Mensch nimmt an allen drei Sphären teil, ohne  
    überzugreifen.  
    """  
  
    ziel_sphaere: ClassVar[Sphaere | None] = None  
    von: Id  
    runde: int = Field(ge=0)  
    als_rolle: Rolle = Rolle.BUERGER  
  
class Zahlen(Absicht):  
    art: Literal["zahlen"] = "zahlen"  
    an: Id  
    halme: Halme  
    zweck: str = ""  
  
class Schenken(Absicht):  
    art: Literal["schenken"] = "schenken"  
    an: Id  
    halme: Halme
```

```

class Abstimmen(Absicht):
    art: Literal["abstimmen"] = "abstimmen"
    antrag: Id
    stimme: Literal["ja", "nein", "enthaltung"]

class BedarfMelden(Absicht):
    art: Literal["bedarf_melden"] = "bedarf_melden"
    produkt: Id
    menge: float = Field(gt=0)

class VorhabenEinbringen(Absicht):
    """Leihgeld suchen. Knapp ist nicht der Betrag,
    sondern die lange Frist."""

    ziel_sphaere: ClassVar[Sphaere | None] =
Sphaere.WIRTSCHAFT
    art: Literal["vorhaben"] = "vorhaben"
    halme: Halme
    mindestfrist_jahre: float = Field(gt=0)
    titel: str = ""

class Importieren(Absicht):
    ziel_sphaere: ClassVar[Sphaere | None] =
Sphaere.WIRTSCHAFT
    art: Literal["importieren"] = "importieren"
    euro_stelle: Id
    produkt: Id
    menge: float = Field(gt=0)

# --- die drei gestaltenden Handlungen: je eine pro
Sphäre ---

class PreisVorschlagen(Absicht):

```

```

        ziel_sphaere: ClassVar[Sphaere | None] =
Sphaere.WIRTSCHAFT
        art: Literal["preis_vorschlagen"] =
"preis_vorschlagen"
        evg: Id
        produkt: Id
        preis_je_einheit: Halme

class RegelBeantragen(Absicht):
    ziel_sphaere: ClassVar[Sphaere | None] =
Sphaere.RECHT
    art: Literal["regel_beantragen"] =
"regel_beantragen"
    rechtsgemeinschaft: Id
    regel: Rahmenregel

class InhaltBestimmen(Absicht):
    """Was gelehrt, erforscht, gestaltet wird."""

    ziel_sphaere: ClassVar[Sphaere | None] =
Sphaere.KULTUR
    art: Literal["inhalt_bestimmen"] =
"inhalt_bestimmen"
    kulturstaette: Id
    inhalt: str

class UmSchenkungWerben(Absicht):
    ziel_sphaere: ClassVar[Sphaere | None] =
Sphaere.KULTUR
    art: Literal["werben"] = "werben"
    kulturstaette: Id
    botschaft: str

AbsichtBeliebig = Annotated[
    Union[

```

```

        Zahlen, Schenken, Abstimmen, BedarfMelden,
        VorhabenEinbringen, Importieren,
        PreisVorschlagen, RegelBeantragen,
        InhaltBestimmen, UmSchenkungenWerben,
    ],
    Field(discriminator="art"),
]

```

```

class AbsichtsPaket(Model):
    """Was ein Client pro Runde an den Server
    schickt."""

    person: Id
    runde: int = Field(ge=0)
    absichten: list[AbsichtBeliebig] = []

```

Wie erkennt die Welt einen Übergriff?

Eine Funktion, wenige Zeilen. Rolle gegen Zielsphäre, fertig.

```

class Uebergriff(Model):
    """Ein Organ handelt mit der Kraft seiner Sphäre
    in einer fremden."""

    runde: int
    person: Id
    quelle: Sphaere
    ziel: Sphaere
    absicht: str

    @property
    def zelle(self) -> str:
        return f"{self.quelle.value}->
        {self.ziel.value}"

    def klassifiziere(a: Absicht) -> Uebergriff | None:

```

```

    """Rolle × gestaltende Handlung ergibt genau die
    3×3-Übergriffs-Matrix.

    Diagonale = gesund. Die sechs Zellen daneben =
    Übergriffe.
    Erlaubt sind sie trotzdem: Die Folgen zeigt die
    Simulation.
    """
    quelle = ROLLEN_SPHAERE[a.als_rolle]
    ziel = type(a).ziel_sphaere
    if quelle is None or ziel is None or quelle is
    ziel:
        return None
    return Uebergriff(runde=a.runde, person=a.von,
    quelle=quelle, ziel=ziel,
                        absicht=type(a).__name__)

```

Wie ist eine Runde gebaut?

Runden, Phasen, Jahreszeiten, und die Kennzahlen, die nach jeder Runde für alle sichtbar werden.

```

class Jahreszeit(str, Enum):
    FRUEHLING = "frühling"
    SOMMER = "sommer"
    HERBST = "herbst"
    WINTER = "winter"

class Phase(str, Enum):
    NATUR = "natur" # Welt: Ernte,
    Alterung, Auffrischung
    ABSICHTEN = "absichten" # Menschen und
    Bots geben Absichten ab
    VERHANDLUNG = "verhandlung" # EVG einigen
    sich auf Preise
    ABSTIMMUNG = "abstimmung" #
    Rechtsgemeinschaft entscheidet

```

```

    AUFLOESUNG = "aufloesung"      # Welt prüft,
    bucht, klassifiziert Übergriffe
    RUECKSCHAU = "rueckschau"      # Kennzahlen und
    Folgen werden sichtbar

class Runde(Modell):
    nr: int = Field(ge=0)
    phase: Phase = Phase.NATUR

    @property
    def jahr(self) -> int:
        return self.nr // 4 + 1

    @property
    def jahreszeit(self) -> Jahreszeit:
        return list(Jahreszeit)[self.nr % 4]

    @property
    def erster_tag(self) -> int:
        return self.nr * TAGE_JE_RUNDE

class Kennzahlen(Modell):
    """Die Wahrnehmungsorgane des Spiels - für alle
    sichtbar."""

    runde: int
    deckung_lebensunterhalt: float = Field(ge=0,
    le=1)
    preisabweichung_median: float
    geldmenge: Halme
    schenkgeld_zur_kultur: Halme
    fristen_mismatch_tage: int = Field(ge=0)
    uebergriffe: dict[str, int] = {} # Matrixzelle
    -> Anzahl in dieser Runde
    beteiligung_abstimmung: float = Field(ge=0,
    le=1)

```

Was sieht ein Spieler?

Nur das eigene Konto, die offenen Anträge, die Preise und die öffentlichen Kennzahlen. Fremde Buchungen bleiben verborgen, wie es das Korn-Protokoll auch für die echte Währung vorsieht.

```
class KontoSicht(Modell):
    """Nur das eigene Konto. Fremde Buchungen
    bleiben verborgen, wie im Korn-Protokoll."""

    gesamt: Halme
    nach_verwendung: dict[Verwendung, Halme]
    davon_sterbend: Halme
    naechster_ablauf_tag: int | None

class Antrag(Modell):
    id: Id
    regel: Rahmenregel
    antragsteller: Id
    als_rolle: Rolle
    offen_bis_runde: int

class Sicht(Modell):
    """Was der Server einem Client pro Phase
    schickt."""

    runde: Runde
    person: Id
    rolle: Rolle
    konto: KontoSicht
    offene_antraege: list[Antrag] = []
    preise: list[Preisvereinbarung] = []
    kennzahlen: Kennzahlen | None = None
    meldungen: list[str] = []
```

Wie wird eine Mission beschrieben?

Eine Mission ist ein Szenario mit einer Frage. Beim Laden prüft sie ihren eigenen Zusammenhang: Gibt es jedes genannte Organ? Hat jede Region eine Rechtsgemeinschaft? Gibt es mindestens einen anerkannten Naturbetrieb, damit Korn überhaupt aufgefrischt werden kann?

```
class Schock(Modell):
    naturgut: Id | Literal["*"]
    ab_runde: int = Field(ge=0)
    dauer_runden: int = Field(4, gt=0)
    faktor: float = Field(ge=0)

class Mission(Modell):
    """Ein Szenario als Spielmission: Welt,
    Menschen, Organe, Schocks - und eine Frage."""

    id: Id
    titel: str
    frage: str
    runden: int = Field(gt=0)
    seed: int = 42
    korn: KornRegeln = KornRegeln()
    regionen: list[Region] = Field(min_length=1)
    produkte: list[Produkt] = []
    personen: list[Person] = Field(min_length=1)
    evgs: list[EVG] = []
    naturbetriebe: list[Naturbetrieb] =
Field(min_length=1)
    rechtsgemeinschaften: list[Rechtsgemeinschaft] =
Field(min_length=1)
    kulturstaetten: list[Kulturstaette] = []
    schocks: list[Schock] = []
    startkorn: dict[Id, Halme] = {} # Halter ->
Halme

    @model_validator(mode="after")
    def _zusammenhang(self) -> "Mission":
```

```

        fehler: list[str] = []
        organe = [*self.evgs, *self.naturbetriebe,
                  *self.rechtsgemeinschaften,
                  *self.kulturstuetten]
        alle_ids = [p.id for p in self.personen] +
        [o.id for o in organe]
        doppelt = {i for i in alle_ids if
        alle_ids.count(i) > 1}
        if doppelt:
            fehler.append(f"doppelte Ids:
        {sorted(doppelt)}")

        regionen = {r.id for r in self.regionen}
        personen = {p.id for p in self.personen}
        organ_sphaere = {o.id: o.sphaere for o in
        organe}
        produkte = {p.id for p in self.produkte}
        gueter = {g.name for r in self.regionen for
        g in r.naturgrundlage.gueter}

        for x in [*self.personen, *organe]:
            if x.region not in regionen:
                fehler.append(f"{x.id}: unbekannte
        Region {x.region}")
            for p in self.personen:
                for m in p.mitgliedschaften:
                    if m.organ not in organ_sphaere:
                        fehler.append(f"{p.id}:
        unbekanntes Organ {m.organ}")
                    elif organ_sphaere[m.organ] is not
        m.sphaere:
                        fehler.append(f"{p.id}:
        {m.organ} gehört nicht zur Sphäre
        {m.sphaere.value}")
            for e in self.evgs:
                for pid in e.mitglieder - personen:
                    fehler.append(f"{e.id}: unbekannte
        Person {pid}")
                for prod in set(e.produkte) - produkte:

```

```

        fehler.append(f"{e.id}: unbekanntes
Produkt {prod}")
    for n in self.naturbetriebe:
        if n.naturgut not in gueter:
            fehler.append(f"{n.id}: Naturgut
{n.naturgut} fehlt in der Naturgrundlage")
        if not any(n.anerkannt for n in
self.naturbetriebe):
            fehler.append("kein anerkannter
Naturbetrieb - Korn könnte nie aufgefrischt werden")
    for r in self.regionen:
        if not any(rg.region == r.id for rg in
self.rechtsgemeinschaften):
            fehler.append(f"Region {r.id} ohne
Rechtsgemeinschaft")
    for halter in self.startkorn:
        if halter not in alle_ids:
            fehler.append(f"Startkorn für
unbekannten Halter {halter}")
    for s in self.schocks:
        if s.naturgut != "*" and s.naturgut not
in gueter:
            fehler.append(f"Schock auf
unbekanntes Naturgut {s.naturgut}")
    if fehler:
        raise ValueError("; ".join(fehler))
    return self

```

Wie sieht eine Mission in YAML aus?

Ein Auszug aus `missionen/erste-ernte.yaml`. Sechs Menschen im Leokiez, ein Hof, eine EVG für Brot, eine freie Schule. Ab Runde 5 kommt ein Dürrejahr.

```

id: erste-ernte
titel: Erste Ernte im Leokiez
frage: >
    Findet die EVG einen Brotpreis nahe am gerechten

```

```
Preis -
  und bleibt genug Schenkgeld für die Schule übrig?
runden: 12          # drei Jahre, eine Runde je
Jahreszeit
seed: 7

regionen:
- id: leokiez
  name: Leokiez, Berlin-Wedding
  naturgrundlage:
    gueter:
      - { name: getreide, normalertrag: 12,
gewicht: 0.7 }
      - { name: holz, normalertrag: 4, gewicht:
0.3 }
    euro_stelle: { id: euro-leokiez, euro_vorrat:
5000 }

personen:
# ... anna, ben, cem ...
- id: dora
  name: Dora (Lehrerin)
  region: leokiez
  spielt_rolle: kultur
  beduerfnisse: { lebensunterhalt_je_runde: 91000
}
  mitgliedschaften:
    - { organ: schule-leokiez, sphaere: kultur,
funktion: lehrerin }
    - { organ: evg-brot, sphaere: wirtschaft,
funktion: verwenderin }
    - { organ: rg-leokiez, sphaere: recht }
# ... emil, frieda ...

evgs:
- id: evg-brot
  name: EVG Brot
  region: leokiez
  produkte: [getreide, brot]
```

```
erzeuger: [anna, ben]
haendler: [cem]
verwender: [dora, emil, frieda]

schocks:
- { naturgut: getreide, ab_runde: 5, dauer_runden:
4, faktor: 0.5 } # Dürrejahr
```

Was ist der erste Schritt?

Noch ganz ohne Grafik. Ein Mensch spielt im Terminal die EVG, alle anderen Rollen sind Bots. Zehn Runden.

Ist das schon spannend, lohnt sich Godot. Ist es das nicht, hilft auch keine Grafik. Dann muss erst die Spielmechanik besser werden.

Die Reihenfolge danach:

1. Welt-Schleife mit Mesa, Korn-Ledger aus `korn-sim` anbinden.
2. Einfache Bots für jede Rolle.
3. Terminal-Client für eine menschliche Rolle.
4. FastAPI-Server, damit mehrere Menschen gleichzeitig spielen.
5. Godot-Client.
6. Zweite Region und Weltrechnung.

Welche Fragen sind noch offen?

- **Wie verhandelt eine EVG?** Das ist der Kern der Spielmechanik und noch nicht entworfen. Angebot und

Bedarf offenlegen, dann Vorschlag und Gegenvorschlag?
Oder ein gemeinsamer Tisch mit Einsicht in die Lage aller?

- **Gibt es ein Spielende mit Bewertung?** Oder nur Befunde? Ein gemeinsames Ziel für alle, die Gesundheit des sozialen Ökosystems, würde das Spiel kooperativ machen.
- **Sollen verdeckte Übergriffe erkannt werden?** Also Wirtschaftsinteressen, die über die Bürgerrolle eingeschleust werden.
- **Wie wächst die Geldmenge mit der Zahl der Menschen?** Auffrischung ist eins zu eins. Für Wachstum braucht es eine zweite Emissionsregel.
- **Wie wird die Zwanziger-Regel begründet?** Im Spiel ist sie ein Schalter. Die Simulation könnte zeigen, was bei 10, 20 oder 50 Mitgliedern passiert.
- **Mensch oder Sprachmodell als Bot?** Bots mit Sprachmodell könnten lebendiger verhandeln, machen aber jeden Lauf teuer und nicht wiederholbar.

Was meinst du?

Das ist ein erster Entwurf. Die Rollen, die Übergriffs-Matrix und die Kennzahlen sind Vorschläge, keine Festlegungen.

Besonders interessiert mich: Stimmt die Zuordnung der drei gestaltenden Handlungen zu den drei Sphären? Und passt die Beispielbelegung der Matrix zur Doppelmatrix?

Rückmeldungen jeder Art sind willkommen.